

2 Related Work

LLM agents in multi-turn environment. LLM-based agents have evolved from handling single-turn queries to serving as autonomous agents capable of multi-turn interactions such as web navigation [60, 70] and complex research [69]. To enable such capabilities, Yao et al. [61] introduced the ReAct (*i.e.*, Reason + Act) framework, which enhances LLMs’ ability to interact with external environments by interleaving reasoning and action. Building on this reasoning-acting prompting paradigm, subsequent works have explored ways to improve agent performance through natural language feedback, enabling iterative refinement [45, 32]. Recently, inference-time scaling has emerged as a promising direction for enabling complex reasoning, with prior research incorporating evaluators (*e.g.*, verifier, reward model) [46, 31] or world models [9]. In addition, there are two major

lines of training approaches: (1) behavior cloning (BC), which involves imitating expert trajectories to guide agent behavior by supervised fine-tuning (SFT) [62, 16, 12], and (2) reinforcement learning (RL), which optimizes agent policies by incentivizing desirable outcomes through rewards [47, 5, 39]. These methods aim to align the agents’ behaviors with task objectives, enabling more robust and generalizable performance.

Memory management for LLM agents. A widely adopted approach to memory management in LLM-based agent systems involves appending all prior information, such as observations, intermediate thoughts, and actions, into the prompt at each interaction turn [61]. While this method is straightforward and effective when the number of interactions required is small, it results in unbounded context growth, leading to linearly scaled inference memory. Moreover, long contexts often contain irrelevant or redundant information, which impairs the model’s reasoning capabilities [2, 30, 56]. To mitigate these issues, recent studies have proposed external memory frameworks, including retrieval-augmented generation and summarization modules [63, 29, 13, 57]. However, these methods are typically trained or used independently of the agent’s policy, creating a disconnect between memory and the reasoning process. In addition, managing and integrating such modules often incurs extra computational overhead and system complexity. Despite these advancements, many RL approaches for training LLM agents still rely on accumulating the full interaction history as memory [24, 69, 39], leaving memory management during training an underexplored area. In this work, we seek to bridge this gap by tightly integrating memory with the agent’s reasoning process, thereby enabling more efficient and context-aware decision-making.